



**RREPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE TELECOMUNICACIONES E
INFORMATICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA
Programación**

**Evaluación 2:
Desarrollo de mi aplicación
con Ionic y Angular**

Docente:

Carlos Márquez.

Autor:

Vladimir Alejandro Rodríguez Hernández
C.I.: 29.915.065

Valencia, diciembre 2025

1. Introducción

Esta evaluación consiste en el desarrollo de una aplicación móvil utilizando el framework Ionic junto con Angular. La aplicación cuenta con tres menús principales: Inicio, Información Personal y Contacto. El objetivo es demostrar el dominio de los conceptos fundamentales de desarrollo móvil híbrido, incluyendo componentes standalone, servicios compartidos, navegación mediante routing y formularios reactivos.

En esta segunda entrega se incorporó una funcionalidad adicional a la página de Contacto: la eliminación de mensajes enviados mediante el método `eliminar()`, el cual hace uso del método `splice()` de JavaScript sobre el arreglo `mensajesEnviados`.

2. Tecnologías Utilizadas

- Ionic Framework: Framework para desarrollar aplicaciones móviles híbridas con tecnologías web.
 - Angular (Standalone): Framework de Google para construir aplicaciones web modernas sin NgModules.
 - TypeScript: Lenguaje con tipado estático basado en JavaScript.
 - Node.js y npm: Entorno de ejecución y gestor de paquetes del proyecto.
 - VS Code: Editor de código utilizado para desarrollar el proyecto.
-

3. Proceso de Instalación

3.1 Instalación del CLI de Ionic

```
npm install -g @ionic/cli
```

Este comando descarga e instala todas las herramientas necesarias para crear y gestionar proyectos Ionic desde la línea de comandos.

3.2 Creación del Proyecto

```
ionic start miApp sidemenu --type=angular
```

Se seleccionó la arquitectura Standalone para no requerir NgModules, simplificando la estructura del proyecto.

3.3 Generación de Páginas

```
cd miApp
ionic generate page pages/inicio
```

```
ionic generate page pages/informacion
ionic generate page pages/contacto
```

4. Estructura del Proyecto

```
src/
  app/
    pages/
      inicio/      → Página de bienvenida y registro
      informacion/ → Página de perfil del estudiante
      contacto/    → Página de formulario de contacto
    services/
      estudiante.ts → Servicio compartido de datos
    app.component.html → Menú lateral
    app.component.ts   → Lógica del menú
    app.routes.ts      → Rutas de navegación
```

5. Descripción de los Archivos Principales

5.1 app.routes.ts — Rutas de Navegación

Define las rutas de la aplicación con `loadComponent` para cargar cada página de forma lazy, mejorando el rendimiento inicial.

```
export const routes: Routes = [
  { path: '', redirectTo: 'inicio', pathMatch: 'full' },
  { path: 'inicio', loadComponent: () =>
    import('./pages/inicio/inicio.page').then(m => m.InicioPage) },
  { path: 'informacion', loadComponent: () =>
    import('./pages/informacion/informacion.page').then(m =>
m.InformacionPage) },
  { path: 'contacto', loadComponent: () =>
    import('./pages/contacto/contacto.page').then(m => m.ContactoPage) }
];
```

5.2 services/estudiante.ts — Servicio Compartido

Actúa como almacén central de datos del estudiante. Al usar `@Injectable({ providedIn: 'root' })`, Angular crea una sola instancia compartida entre todas las páginas.

```
@Injectable({ providedIn: 'root' })
export class EstudianteService {
  datos = { nombre: '', carrera: '', correo: '', telefono: '' };
  guardar(nombre, carrera, correo, telefono) { ... }
  estaRegistrado(): boolean { return this.datos.nombre.trim() !== ''; }
}
```

5.3 app.component.ts — Componente Raíz

```
export class AppComponent {
  constructor(
    private menu: MenuController,
    public estudianteService: EstudianteService
  ) {}
  cerrarMenu() { this.menu.close(); }
}
```

6. Páginas de la Aplicación

6.1 Página de Inicio

Muestra un formulario de registro antes del registro y una tarjeta de bienvenida después. El método registrar() valida los campos antes de guardar en el servicio compartido.

```
registrar() {
  if (!this.nombre || !this.carrera || !this.correo || !this.telefono) {
    alert('Por favor completa todos los campos.');
```

6.2 Página de Información Personal

Lee los datos del EstudianteService y los muestra en una tarjeta visual. Usa el operador || para mostrar 'Sin registrar' cuando un campo está vacío.

```
{{ estudianteService.datos.carrera || 'Sin registrar' }}
```

6.3 Página de Contacto — Formulario y Mensajes

La página de contacto tiene un formulario con tres campos (nombre, correo y mensaje) que usa two-way data binding mediante [(ngModel)] para sincronizar en tiempo real los campos con las variables del componente.

Al enviar, el método enviar() valida que los campos no estén vacíos, agrega el mensaje al arreglo mensajesEnviados y limpia el formulario:

```
enviar() {  
  if (!this.nombre || !this.correo || !this.mensaje) {  
    alert('Por favor completa todos los campos.');    return;  
  }  
  this.mensajesEnviados.push({  
    nombre: this.nombre, correo: this.correo, mensaje: this.mensaje  
  });  
  this.nombre = ''; this.correo = ''; this.mensaje = '';  
}
```

7. Funcionalidad Nueva: Eliminar Mensaje

7.1 Descripción

Se agregó la función eliminar() que permite borrar un mensaje específico de la lista de mensajes enviados. El usuario presiona el botón rojo 'Eliminar mensaje' en cada tarjeta y ese registro desaparece de la pantalla de inmediato.

Si se elimina el último mensaje, la sección 'Mensajes enviados' desaparece completamente gracias al *ngIf que ya existía en el template.

7.2 Código en contacto.page.ts

Se agregó el método eliminar() dentro de la clase ContactoPage, después del método enviar() existente:

```
// Recibe el índice del mensaje a borrar desde el *ngFor del HTML.  
// splice(indice, 1) elimina exactamente 1 elemento en esa posición del  
arreglo.  
eliminar(indice: number) {  
  this.mensajesEnviados.splice(indice, 1);  
}
```

El parámetro indice: number es el número de posición del mensaje dentro del arreglo mensajesEnviados. El método splice(indice, 1) es una función nativa de JavaScript que elimina 1 elemento exactamente en esa posición, desplazando automáticamente los demás elementos.

7.3 Código en contacto.page.html

En el template HTML se agregó el botón dentro del bloque `*ngFor` que itera sobre cada mensaje. La clave es que el índice `i`, declarado en `let i = index`, se pasa directamente al método `eliminar(i)`:

```
<!-- *ngFor ya existía: genera una tarjeta por cada mensaje -->
<div *ngFor="let m of mensajesEnviados; let i = index">

  <p>{{ m.mensaje }}</p>

  <!-- BOTÓN NUEVO: pasa el índice i para saber cuál mensaje borrar -->
  <ion-button
    expand="block"
    (click)="eliminar(i)"
    style="--background:#c0392b; --border-radius:8px;
      --color:white; height:36px; margin-top:10px;">
    <ion-icon name="trash-outline" slot="start"></ion-icon>
    Eliminar mensaje
  </ion-button>

</div>
```

7.4 Relación entre el HTML y el TypeScript

El flujo completo de la función es el siguiente:

- El `*ngFor` itera el arreglo y expone el índice `i` de cada elemento con `let i = index`.
- El `(click)="eliminar(i)"` en el botón llama al método del componente, pasando ese índice.
- El método `eliminar(indice)` usa `splice(indice, 1)` para borrar el elemento en esa posición.
- Angular detecta el cambio en el arreglo y actualiza la vista automáticamente (sin recargar la página).
- Si el arreglo queda vacío, el `*ngIf="mensajesEnviados.length > 0"` oculta toda la sección.

8. Conceptos Clave Aplicados

8.1 Componentes Standalone

Cada componente declara sus propias dependencias en el array `imports` del decorador `@Component`. Esto hace cada componente independiente y elimina la necesidad de un `NgModule` central.

8.2 Inyección de Dependencias

Angular crea automáticamente una sola instancia del `EstudianteService` y la inyecta en los componentes que la soliciten. Esto garantiza que todos los componentes comparten los mismos datos sin pasarlos manualmente entre páginas.

8.3 Two-Way Data Binding

El uso de `[(ngModel)]` en los campos del formulario crea un enlace bidireccional: cuando el usuario escribe, la variable del componente se actualiza, y viceversa.

8.4 Directivas `*ngIf` y `*ngFor`

`*ngIf` muestra u oculta elementos según una condición booleana. `*ngFor` itera sobre arreglos generando un elemento HTML por cada ítem, exponiendo también el índice con `let i = index`, que fue fundamental para la función `eliminar()`.

8.5 Manipulación de Arreglos con `splice()`

El método `splice(indice, cantidad)` de JavaScript permite eliminar elementos de un arreglo en una posición específica. Al modificar el arreglo `mensajesEnviados`, Angular detecta el cambio y actualiza la vista de forma reactiva sin necesidad de código adicional.

8.6 Routing con `loadComponent`

La navegación usa lazy loading con `loadComponent`, cargando el código de cada página solo cuando el usuario navega hacia ella, reduciendo el tiempo de carga inicial.

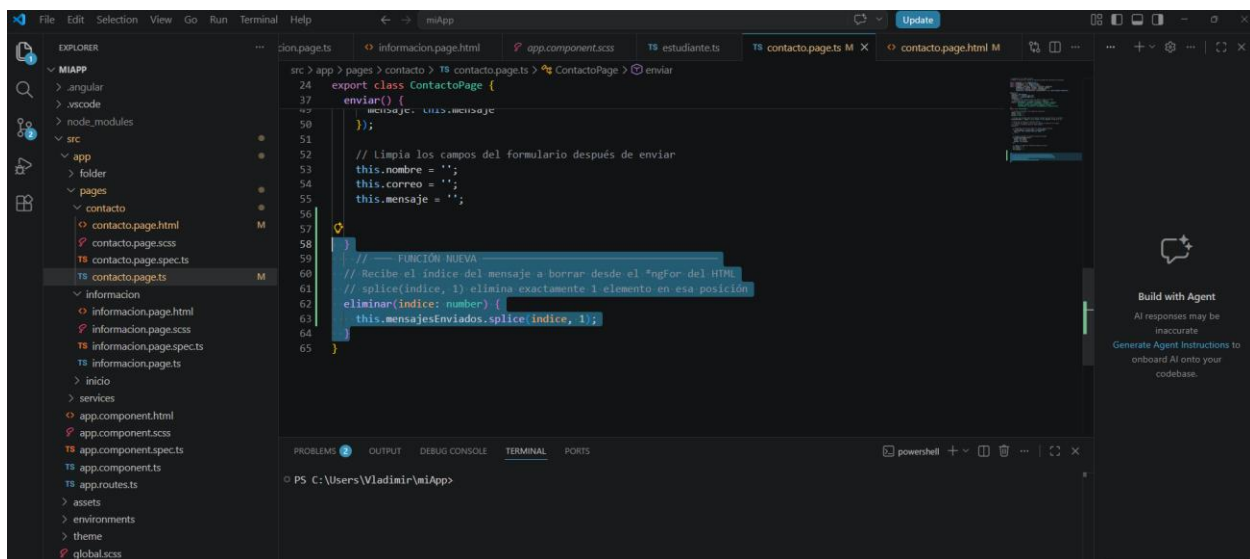
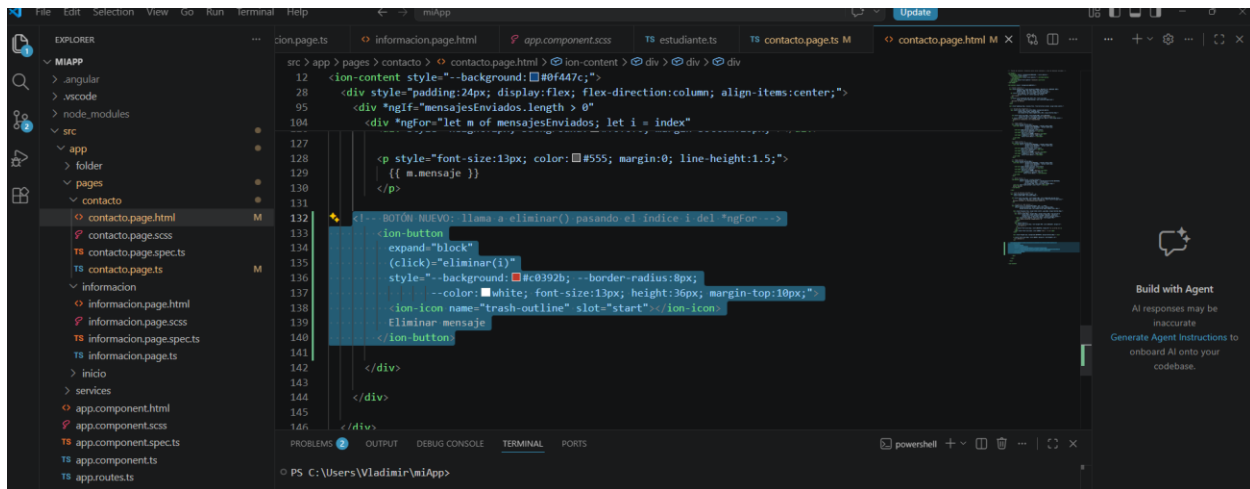
9. Conclusiones

El desarrollo de esta aplicación permitió aplicar de forma práctica los conceptos fundamentales de Ionic y Angular. Se comprendió la importancia de la arquitectura standalone, el uso de servicios compartidos para comunicar datos entre páginas, y el sistema de routing para la navegación.

La incorporación de la función `eliminar()` en la página de Contacto demostró cómo Angular permite manipular arreglos en el componente TypeScript y reflejar esos cambios automáticamente en la vista HTML, sin recargas ni lógica adicional de renderizado. El índice expuesto por `*ngFor` fue la pieza clave que conectó el botón de cada tarjeta con el elemento exacto del arreglo a eliminar.

El resultado es una aplicación funcional con diseño profesional de tonos navales, menú lateral, formularios con validación, visualización de datos en tiempo real y gestión dinámica de la lista de mensajes.

10. Anexos



Vladimir Rodríguez
C.I. 29.915.065